

Traceability Matrix

QUBO-Based Sensor Node Placement for Device-Free Localization

Mohamed Khalil Brik

Spring 2026

Contents

1	Introduction	2
2	Document References	2
3	Requirement Descriptions	2
4	Requirements to Implementation Traceability	2
5	Requirements to Test Case Traceability	2
6	Full Traceability Matrix	2
7	Coverage Summary	7

Introduction

This document provides the full traceability matrix for the QUBO-based sensor node placement system. It links every functional requirement from the System Requirements and Design Specifications to the implementation component that fulfills it and the test case in the Verification and Validation Report that verifies it.

The matrix gives reviewers a single reference point to confirm that no requirement is left unimplemented or untested.

Document References

Abbreviation	Document
SRS	System Requirements and Design Specifications
ESC	Engineering Standards and Constraints
VVR	Verification and Validation Report

Requirement Descriptions

The table below summarizes each functional requirement for quick reference.

Requirements to Implementation Traceability

Requirements to Test Case Traceability

Full Traceability Matrix

The table below consolidates all three dimensions: requirements, implementation components, and test cases.

ID	Description
FR-1	Load all input CSV files (RSS matrices, coordinates, pairs, importance, redundancy).
FR-2	Build QUBO matrix Q parameterized by α using importance and redundancy values.
FR-3	Run binary search over $\alpha \in [0, 1]$ with tolerance $\epsilon = 0.01$ to find a budget-feasible selection.
FR-4	Solve the QUBO using SA (SASampler) and SQA (SQASampler) with specified read and sweep counts.
FR-5	Implement a greedy baseline that selects pairs by descending importance while respecting the same AP and MP budgets.
FR-6	Evaluate any selection using 3-NN localization and report the mean Euclidean error in meters over the test set.
FR-7	Compute the Time-to-Solution (TTS) at $p_R = 0.99$ for SA and SQA.
FR-8	Repeat the full binary search 100 times with varying seeds and report mean, standard deviation, min, and max statistics.
FR-9	Save all numerical results to data/output/ as CSV files.
FR-10	Generate all ten publication-quality figures and save them to figures/ at 300 DPI.

Table 1: Functional requirement summary.

Requirement	Implementing Component	File(s)
FR-1	CSV loading with <code>pd.read_csv</code> ; shape printed to console for runtime verification.	<code>scripts/01</code> , <code>scripts/04</code>
FR-2	<code>build_qubo(alpha, I, R)</code> function constructs the sparse QUBO dictionary.	<code>scripts/01</code> , <code>scripts/04</code>
FR-3	<code>binary_search()</code> function; <code>while b - a >= EPSILON</code> loop with feasibility check.	<code>scripts/01</code> , <code>scripts/04</code>
FR-4	<code>oj.SASampler().sample_qubo()</code> with 100 reads / 1000 sweeps; <code>oj.SQASampler().sample_qubo()</code> with 10 reads / 1000 sweeps.	<code>scripts/01</code> , <code>scripts/04</code>
FR-5	Greedy loop over importance-sorted pairs; set-based AP/MP budget check.	<code>scripts/03</code> , <code>scripts/04</code>
FR-6	<code>localizer()</code> / <code>knn_error()</code> function; Euclidean distance in RSS space; mean error over test set.	<code>scripts/01</code> , <code>scripts/03</code> , <code>scripts/04</code>
FR-7	<code>calculate_tts(p_s, tau, p_target)</code> function; τ from wall-clock time (SA) or $20 \mu s$ (SQA).	<code>scripts/01</code> , <code>scripts/04</code>
FR-8	Outer loop over <code>range(N_RUNS)</code> with <code>seed = run</code> ; <code>pandas</code> groupby statistics saved to summary CSV.	<code>scripts/04</code>
FR-9	<code>df.to_csv()</code> calls at the end of each script save seven output files.	<code>scripts/01</code> , <code>scripts/03</code> , <code>scripts/04</code>
FR-10	<code>matplotlib</code> figures with IEEE <code>rcParams</code> ; saved via <code>fig.savefig()</code> .	<code>scripts/02</code> , <code>scripts/05</code> , <code>scripts/06</code>

Table 2: Requirements to implementation traceability.

Requirement	Test Case(s)	What is Verified
FR-1	TC-01	Array shapes match expected dimensions.
FR-2	TC-02	QUBO diagonal and off-diagonal terms at $\alpha = 0$ and $\alpha = 1$.
FR-3	TC-03, TC-04	Search terminates within 7 iterations; all feasible solutions are within budget.
FR-4	TC-03, TC-04	Solver is called correctly; budget check applied to its output.
FR-5	TC-08, TC-10	Greedy is deterministic; QUBO accuracy is no worse than greedy.
FR-6	TC-05, TC-06	Correct error for full selection; infinity for empty selection.
FR-7	TC-07	Correct TTS at $p_s = 0$ and $p_s \geq 1$ boundary cases.
FR-8	TC-11, TC-12, TC-14	Statistics are consistent; row count is correct; budget holds in all 100 trials.
FR-9	TC-09	All seven output files exist and are non-empty.
FR-10	TC-13	All figures saved at 300 DPI.

Table 3: Requirements to test case traceability.

Req.	Description (short)	Implementing Script	Key Function / Component	Test Cases
FR-1	Data ingestion	01, 04	<code>pd.read_csv</code> , <code>shape</code> checks	TC-01
FR-2	QUBO construction	01, 04	<code>build_qubo()</code>	TC-02
FR-3	Binary search	01, 04	<code>binary_search()</code> loop	TC-03, TC-04
FR-4	SA / SQA solvers	01, 04	<code>SASampler</code> , <code>SQASampler</code>	TC-03, TC-04
FR-5	Greedy baseline	03, 04	Importance-sorted greedy loop	TC-08, TC-10
FR-6	k-NN evaluation	01, 03, 04	<code>localizer()</code> / <code>knn_error()</code>	TC-05, TC-06
FR-7	TTS computation	01, 04	<code>calculate_tts()</code>	TC-07
FR-8	Multi-run experiment	04	100-run loop with <code>seed = run</code>	TC-11, TC-12, TC-14
FR-9	Result persistence	01, 03, 04	<code>df.to_csv()</code> calls	TC-09
FR-10	Figure generation	02, 05, 06, 07	<code>matplotlib</code> with <code>IEEE rcParams</code>	TC-13

Table 4: Full requirements–implementation–test traceability matrix.

Coverage Summary

- **Requirements coverage:** 10 of 10 functional requirements are implemented (100%).
- **Test coverage:** 10 of 10 functional requirements are covered by at least one test case (100%).
- **Test pass rate:** 14 of 14 test cases pass (100%).
- **NFR coverage:** All five non-functional requirements (NFR-1 through NFR-5) are addressed in the Engineering Standards and Constraints document and verified through code inspection.